

```
import pandas as pd
import matplotlib.pyplot as plt

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("File:", filename)
print("Dataset Shape:", df.shape)
print("Columns:", df.columns.tolist())

display(df.head())
```

Choose Files wine-quality...and-red.csv

wine-quality-white-and-red.csv(text/csv) - 390514 bytes, last modified: 8/17/2026 - 100% done

Saving wine-quality-white-and-red.csv to wine-quality-white-and-red (1).csv

File: wine-quality-white-and-red (1).csv

Dataset Shape: (6497, 13)

Columns: ['type', 'fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol', 'quality', 

	type	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	white	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.0010	3.00	0.45	8.8	6
1	white	6.3	0.30	0.34	1.6	0.049	14.0	132.0	0.9940	3.30	0.49	9.5	6
2	white	8.1	0.28	0.40	6.9	0.050	30.0	97.0	0.9951	3.26	0.44	10.1	6
3	white	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6
4	white	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.9956	3.19	0.40	9.9	6

```
# Clean column names
df.columns = [
    str(col).strip().lower().replace(" ", "_")
    for col in df.columns
]
```

```
# Find target column
if "quality" in df.columns:
    target = "quality"
elif "quality_score" in df.columns:
    target = "quality_score"
```

```
else:
    raise ValueError(
        "Quality target column was not found."
    )

# Remove non-numeric columns
X = df.drop(columns=[target])

X = X.select_dtypes(
    include=["int64", "float64"]
)

y = df[target]

# Remove missing values
data = pd.concat(
    [X, y],
    axis=1
).dropna()

X = data.drop(columns=[target])
y = data[target]

print("Features:", X.shape[1])
print("Samples:", X.shape[0])
print("\nTarget distribution:")
print(y.value_counts().sort_index())
```

Features: 11
Samples: 6497

Target distribution:
quality
3 30
4 216
5 2138
6 2836
7 1079
8 193
9 5
Name: count, dtype: int64

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(
    X_train
)

X_test_scaled = scaler.transform(
```

```
    X_test
)

print("Training samples:", X_train_scaled.shape[0])
print("Testing samples:", X_test_scaled.shape[0])
print("Original features:", X_train_scaled.shape[1])
```

```
Training samples: 4547
Testing samples: 1950
Original features: 11
```

```
# Reduce the dataset to 5 principal components
n_components = min(5, X_train_scaled.shape[1])

pca = PCA(
    n_components=n_components
)

X_train_pca = pca.fit_transform(
    X_train_scaled
)

X_test_pca = pca.transform(
    X_test_scaled
)

pca_model = LogisticRegression(
    max_iter=2000
)

pca_model.fit(
    X_train_pca,
    y_train
)

pca_predictions = pca_model.predict(
    X_test_pca
)

pca_accuracy = accuracy_score(
    y_test,
    pca_predictions
)

print(
    "PCA Components:",
    n_components
)

print(
    "Explained Variance:",
    round(
        pca.explained_variance_ratio_.sum() * 100,
        2
    ),
    "%"
)
```

```
)

print(
    "PCA Accuracy:",
    round(
        pca_accuracy * 100,
        2
    ),
    "%"
)
)
```

PCA Components: 5
Explained Variance: 79.92 %
PCA Accuracy: 51.95 %

```
# Select the 5 most useful original features
k = min(5, X_train_scaled.shape[1])

selector = SelectKBest(
    score_func=f_classif,
    k=k
)

X_train_selected = selector.fit_transform(
    X_train_scaled,
    y_train
)

X_test_selected = selector.transform(
    X_test_scaled
)

selection_model = LogisticRegression(
    max_iter=2000
)

selection_model.fit(
    X_train_selected,
    y_train
)

selection_predictions = (
    selection_model.predict(
        X_test_selected
    )
)

selection_accuracy = accuracy_score(
    y_test,
    selection_predictions
)

selected_features = X.columns[
    selector.get_support()
]
```

```
print(
    "Selected Features:"
)

print(
    list(selected_features)
)

print(
    "\nFeature Selection Accuracy:",
    round(
        selection_accuracy * 100,
        2
    ),
    "%"
)
```

Selected Features:
['volatile_acidity', 'chlorides', 'free_sulfur_dioxide', 'density', 'alcohol']

Feature Selection Accuracy: 53.69 %

```
print("===== FINAL COMPARISON =====")

print(
    "PCA Accuracy:",
    round(pca_accuracy * 100, 2),
    "%"
)

print(
    "Feature Selection Accuracy:",
    round(selection_accuracy * 100, 2),
    "%"
)

if pca_accuracy > selection_accuracy:
    print("\nPCA performed better.")
elif selection_accuracy > pca_accuracy:
    print("\nFeature Selection performed better.")
else:
    print("\nBoth methods achieved the same accuracy.")

# Comparison graph
methods = [
    "PCA",
    "Feature Selection"
]

accuracies = [
    pca_accuracy,
    selection_accuracy
]

plt.figure(figsize=(8, 5))
```

```
plt.bar(
    methods,
    accuracies
)

plt.ylabel("Accuracy")
plt.xlabel("Dimensionality Reduction Method")
plt.title(
    "PCA vs Feature Selection"
)

plt.ylim(0, 1)
plt.grid(
    axis="y",
    alpha=0.3
)

plt.show()
```

===== FINAL COMPARISON =====

PCA Accuracy: 51.95 %

Feature Selection Accuracy: 53.69 %

Feature Selection performed better.



